

ContextFusion: A Provider-Neutral Context Compiler for Token-Efficient and Low-Latency LLM Workflows

Rohan R
Independent Researcher
ORCID: [0009-0005-9225-1775](https://orcid.org/0009-0005-9225-1775)

2026

Abstract

Large language model (LLM) applications often degrade when prompt construction is treated as simple concatenation. The result is token bloat, unstable latency, and poor portability across providers. In this paper, I present *ContextFusion*, an open-source context compiler that turns heterogeneous sources into compact, budget-constrained context packets for chat and agent workloads. The system combines deterministic ingestion, representation generation, utility-risk scoring, multi-objective planning, and provider-aware compilation into one middleware layer. It supports OpenAI, Anthropic, Ollama, and OpenAI-compatible APIs, with extensions for MCP, LangChain, and LlamaIndex style integrations. On repository benchmarks, ContextFusion preserves task success while substantially reducing context tokens, and improves end-to-end API latency in tested Anthropic runs. The implementation is modular, reproducible, and designed for practitioners who need context quality under strict token and latency budgets, with both Python-native CLI access and an npm wrapper endpoint for general users.

Keywords: context engineering; prompt optimization; retrieval-augmented generation; knapsack optimization; latency-aware planning; LLM middleware; agent systems; provider interoperability

1 Introduction

Prompt size has become a practical bottleneck in production LLM systems. Even when models accept large windows, sending unnecessary context increases cost, latency, and failure surface area. In agent loops, this issue compounds because each turn can replay stale or redundant material [7]. This challenge is amplified in transformer-era prompting workflows where context construction has become an explicit systems concern [1, 3].

I built ContextFusion to address this concrete engineering problem: *how do we keep only high-value context while preserving answer quality and model portability?* Rather than coupling optimization logic to a single provider API, ContextFusion is designed as a provider-neutral middleware layer that emits compact, structured context artifacts and then compiles them into provider-specific payloads. The design is motivated by practical RAG and retrieval-reranking tradeoffs in existing literature [4, 5, 6].

2 System Overview

ContextFusion implements the following pipeline:

Ingest → Normalize → Canonical IR → Precompute → Dedup/Fingerprint

→ Retrieve → Rerank → Plan → Compress → Delta Fuse → Compile

The repository is organized around modular packages in `src/context_portfolio_optimizer/`, including:

- `ingestion/`, `normalization/`, `representations/`
- `retrieval/`, `scoring/`, `allocation/`
- `ir/`, `caching/`, `fusion/`, `compression/`
- `providers/`, `agents/`, `mcp_server/`, `integrations/`

2.1 Representative Usage Snippets

Python CLI entrypoint:

Listing 1: Core Python CLI usage (reproducible local run).

```
cpo precompute ./data
cpo run ./data --budget 3000 --query "Summarize architecture" \
  --provider anthropic --model claude-sonnet-4-6 --mode chat
```

NPM wrapper entrypoint for general users:

Listing 2: NPM wrapper flow for non-developer users.

```
npm install -g ./npm-package
contextfusion setup
contextfusion run ./examples/gui_input \
  --query "Summarize key facts" \
  --provider anthropic --model claude-sonnet-4-6
```

3 Optimization Formulation

The original foundation is a knapsack-style objective:

$$\max \sum_i (\text{utility}_i - \text{risk}_i) z_i \quad \text{s.t.} \quad \sum_i \text{tokens}_i z_i \leq B, \quad z_i \in \{0, 1\}. \quad (1)$$

In the current implementation, this is extended to a configurable multi-objective planner:

$$\max \sum_i (w_u \cdot \text{utility}_i - w_r \cdot \text{risk}_i - w_t \cdot \text{token_cost}_i - w_l \cdot \text{latency_cost}_i + w_c \cdot \text{cacheability}_i + w_d \cdot \text{diversity}_i) z_i. \quad (2)$$

To improve selection under tight budgets, each candidate also gets a value-density estimate:

$$\text{value_density}_i = \frac{\text{utility}_i - \text{risk}_i}{\max(\text{tokens}_i, 1)}. \quad (3)$$

The optimization framing follows classical resource-allocation formulations in combinatorial optimization [2, 8].

4 Canonical Intermediate Representation

ContextFusion emits a typed canonical IR centered on:

- **ContextPacket**: selected blocks, citations, budget metadata, cache segments, and output contract.
- **CacheSegment**: stable versus dynamic segments for prompt-cache-aware assembly.
- **ContextDelta**: added/removed/updated/unchanged blocks for incremental agent turns.

This representation decouples core optimization from provider formatting, which keeps the architecture portable across model APIs.

For distribution, ContextFusion is exposed through its Python CLI (`cpo`) and a lightweight npm wrapper package (`@rotsl/contextfusion`) that targets non-developer usage patterns.

5 Provider-Neutral Compilation

Provider adapters currently support OpenAI, Anthropic, Ollama, and OpenAI-compatible endpoints. A packet compiler maps the same selected context into mode-specific formats:

- **QA mode**: evidence-first compact context with citation-friendly packing.
- **Code mode**: signatures, changed regions, and debugging-relevant snippets.
- **Agent mode**: working-memory brief, state deltas, and tool-result minimization.
- **Chat mode**: concise conversational context with provider-specific message layout.

This packaging strategy is consistent with ecosystem-level interoperability goals seen in modern orchestration frameworks [9, 10].

6 Experimental Setup

I report results from repository benchmarks under `benchmarks/`:

- `BENCHMARK_RESULTS.md`: deterministic local evaluation.
- `BENCHMARK_API_RESULTS.md`: Anthropic API comparison (`claude-sonnet-4-6`).

The tiny task dataset is `benchmarks/datasets/tiny_tasks.jsonl`. Success is measured by expected-answer containment in model output (case-insensitive), and token/latency metrics are recorded per run.

7 Results

7.1 Deterministic Local Benchmark

Interpretation: ContextFusion reduces context size substantially while preserving deterministic task success. The planning overhead is visible in local context assembly time, which is expected for budget-aware selection.

7.2 API Benchmark (Anthropic)

Compared with full-context prompting, ContextFusion lowers input payload by over an order of magnitude and yields lower observed total latency in this benchmark setting.

Metric	With ContextFusion	Without ContextFusion
Success rate	100.0%	100.0%
Average context tokens	34.7	99.0
Average token reduction	65.0%	—
Average context latency (ms)	86.5	0.017

Table 1: Local tiny benchmark summary from `BENCHMARK_RESULTS.md`.

Metric	With ContextFusion	Without ContextFusion
Runs	3	3
Success rate	100.0%	100.0%
Average context tokens	10.3	947.0
Average model latency (ms)	7694.6	8609.5
Average total latency (ms)	7763.3	8609.6

Table 2: Anthropic API benchmark summary from `BENCHMARK_API_RESULTS.md`.

8 Discussion

Three practical observations emerge:

1. **Token efficiency is the most consistent gain.** Across local and API runs, compact planning sharply reduces context tokens.
2. **Latency behavior is workload-dependent.** Context selection adds overhead, but this can be offset by lower model-side processing time when prompt payload shrinks.
3. **IR separation improves portability.** A single selection pipeline can target multiple providers and runtime modes without duplicating optimization logic.

9 Future Work

Planned extensions are:

1. **Larger-scale benchmarks.** Add broader task sets with longer contexts and multi-turn trajectories to stress-test latency and token savings under realistic workloads.
2. **Learned reranking and utility modeling.** Add optional learned rerankers on top of the deterministic baseline while preserving reproducibility toggles.
3. **Multimodal extraction improvements.** Improve OCR and image-grounded context quality to reduce “no text detected” failure paths.
4. **End-to-end caching telemetry.** Add explicit cache hit-rate and delta-fusion savings dashboards across repeated agent loops.
5. **Formal evaluation protocols.** Extend beyond containment-based success with factuality and citation-grounding metrics.

10 Limitations

Current benchmarks are intentionally small and controlled. They are useful for regression tracking but not sufficient for broad generalization across all domains, prompts, and model

families. Future work should include larger corpora, longer agent trajectories, and controlled ablations over compression and cache policies.

11 Conclusion

ContextFusion demonstrates that context handling can be treated as a first-class systems problem rather than ad hoc prompt concatenation. By combining deterministic planning, compact representations, and provider-aware compilation, the framework preserves task correctness while significantly reducing token usage and improving practical performance in evaluated settings. This makes it a strong candidate middleware layer for production chat and agent applications.

Reproducibility

The implementation and benchmark artifacts are available in the public repository:

<https://github.com/rotsl/context-fusion>

NPM wrapper package:

<https://www.npmjs.com/package/@rotsl/contextfusion>

Versioned Zenodo record (v0.1.0):

<https://doi.org/10.5281/zenodo.18945681>

Key commands:

Listing 3: Minimal reproducibility commands.

```
make install-dev
cpo precompute ./data
cpo run ./data --budget 3000 --query "Summarize architecture"
npm install -g ./npm-package
contextfusion setup
contextfusion run ./examples/gui_input --query "Summarize" --provider anthropic
```

Funding

No external funding was received for this work.

Conflicts of Interest

The author declares no conflict of interest.

Data and Code Availability

Code, benchmark artifacts, and reproducibility scripts are publicly available in the repository and release archive [11].

References

- [1] Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems.
- [2] Kellerer, H., Pferschy, U., and Pisinger, D. (2004). *Knapsack Problems*. Springer.
- [3] Liu, P., et al. (2023). *Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in NLP*. ACM Computing Surveys.
- [4] Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems.
- [5] Robertson, S., and Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 333–389.
- [6] Carbonell, J., and Goldstein, J. (1998). *The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries*. Proceedings of SIGIR 1998, 335–336.
- [7] Yao, S., Zhao, J., Yu, D., et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. International Conference on Learning Representations (ICLR).
- [8] Karp, R. M. (1972). *Reducibility Among Combinatorial Problems*. In R. E. Miller and J. W. Thatcher (Eds.), *Complexity of Computer Computations*, 85–103.
- [9] Chase, H. (2022). *LangChain*. <https://github.com/langchain-ai/langchain>
- [10] LlamaIndex Project (2022). *LlamaIndex*. https://github.com/run-llama/llama_index
- [11] Rohan, R. (2026). *ContextFusion* (Version 0.1.0) [Software]. Zenodo. <https://doi.org/10.5281/zenodo.18945681>